

[Euron] 머신러닝 스터디 3주차

[통계](#) [수정](#) [삭제](#)[조운서](#) · [방금 전](#)

❤️ 0

교재: 파이썬 머신러닝 완벽 가이드 (권철민)

제 4장. 분류

01. 분류의 개요

- **분류**: 미지의 레이블 값을 예측하는 방법

- 지도학습의 대표적인 예시
- 학습 데이터로 주어진 데이터의 피쳐와 레이블값을 머신러닝 알고리즘으로 학습하여 모델 생성

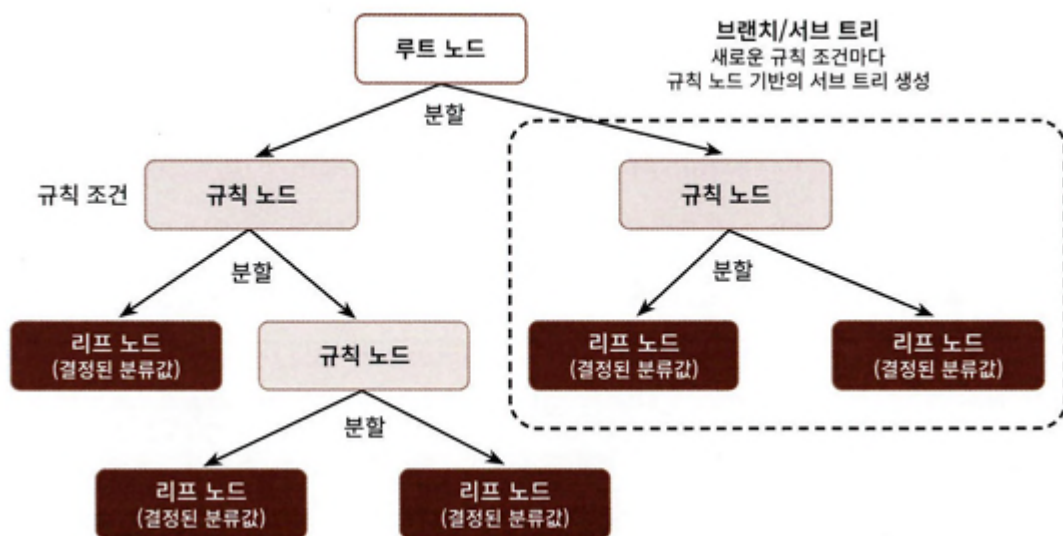
분류 알고리즘	설명
나이브 베이즈	베이즈 통계와 생성 모델 기반
로지스틱 회귀	독립변수와 종속변수의 선형 관계성 기반
결정 트리	데이터 균일도에 따른 규칙 기반
서포트 벡터 머신	개별 클래스 간의 최대 분류 마진을 효과적으로 찾아줌
최소 근접 알고리즘	근접 거리를 기준으로 함

분류 알고리즘	설명
신경망	심층 연결 기반
앙상블	서로 다른(또는 같은) 머신러닝 알고리즘을 결합

결정 트리

- **결정 트리**: 데이터에 있는 규칙을 학습을 통해 자동으로 찾아내 트리 기반의 분류 규칙을 만드는 것

- ML 알고리즘 중 직관적으로 이해하기 쉬움
- if/else 기반
- 어떤 기준을 바탕으로 규칙을 만들지가 성능을 크게 좌우



- 규칙이 많다 = 분류를 결정하는 방식이 더욱 복잡해진다 $\approx$ 과적합 가능성 $\uparrow$ $\approx$ 예측 성능이 저하될 가능성 $\uparrow$

정보 균일도

- 최대한 균일한 데이터 세트를 구성할 수 있도록 분할하는 것이 필요
- 결정 노드는 정보 균일도가 높은 데이터 세트 먼저 선택할 수 있도록 규칙 조건을 만들
- 정보 균일도가 데이터 세트로 쪼개질 수 있도록 조건을 찾아 서브 데이터 세트를 만들고, 다시 이 서브 데이터 세트에서 균일도가 높은 자식 데이터 세트 쪼개는 방식을 자식 트리로 내려가면서 반복하는 방식으로 데이터 값을 예측

- 정보 균일도를 측정하는 대표적인 방법에 엔트로피를 이용한 정보 이득 지수와 지니 계수가 있음
 - **엔트로피** : 데이터 집합의 혼잡도를 의미
 - ↳ 서로 다른 값이 섞여있으면 엔트로피 ↑, 같은 값이 섞여 있으면 엔트로피 ↓
 - **정보 이득**: $1 - (\text{엔트로피 지수})$
 - ↳ 정보 이득이 높은 속성을 기준으로 분할
 - **지니 계수**: 경제학에서 불평등 지수를 나타낼 때 사용하는 계수로, 0이 가장 평등하고 1로 갈수록 불평등
 - ↳ 지니 계수가 낮은 속성을 기준으로 분할

결정 트리 모델의 특징

장점	단점
쉽고 직관적	과적합으로 알고리즘 성능 ↓
피쳐 스케일링이나 정규화 등 사전 가공 영향도 ↓	트리의 크기를 사전에 제한하는 튜닝 필요

결정 트리 파라미터

- 사이킷런은 결정 트리 알고리즘을 구현한 `DecisionTreeRegressor` 클래스를 제공
- 사이킷런의 결정 트리 구현은 CART 알고리즘 기반

파라미터 명	설명
<code>min_samples_split</code>	노드를 분할하기 위한 최소한의 샘플 데이터 수로 과적합을 제어하는 데 사용됨. 디폴트는 2이고 작게 설정할수록 분할되는 노드 ↑
<code>min_samples_leaf</code>	분할이 될 경우 왼쪽과 오른쪽의 브랜치 노드에서 가져야 할 최소한의 샘플 데이터 수. 큰 값으로 설정될수록 노드 분할 ↓ (그러나 비대칭적 데이터의 경우 특정 클래스의 데이터가 극도로 작을 수 있으므로 이 경우는 작게 설정 필요)
<code>max_features</code>	최적의 분할을 위해 고려할 최대 피쳐 개수. 디폴트는 None으로 데이터 세트의 모든 피쳐를 사용해 분할 수행. int형으로 지정 -> 대상 피쳐의 개수, float형으로 지정 -> 전체 피쳐 중 대상 피쳐의 퍼센트

파라미터 명	설명
max_depth	트리의 최대 깊이를 규정. 디폴트는 None으로 완벽하게 클래스 결정 값이 될 때까지 깊이를 계속 키우며 분할하거나 노드가 가지는 데이터 개수가 min_samples_split 보다 작아질 때까지 계속 깊이를 증가시킴
max_leaf_nodes	말단 노드의 최대 개수

결정 트리 모델의 시각화

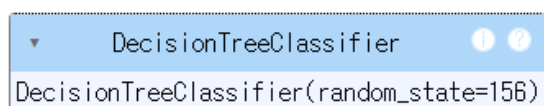
- `export_graphviz()` : Graphviz 패키지 사용

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
import warnings
warnings.filterwarnings('ignore')
```

```
dt_clf = DecisionTreeClassifier(random_state=156)

iris_data = load_iris()
X_train, x_test, y_train, y_test = train_test_split(iris_data.data, iris_data.target,
                                                    test_size=0.2, random_state=11)

dt_clf.fit(X_train, y_train)
```

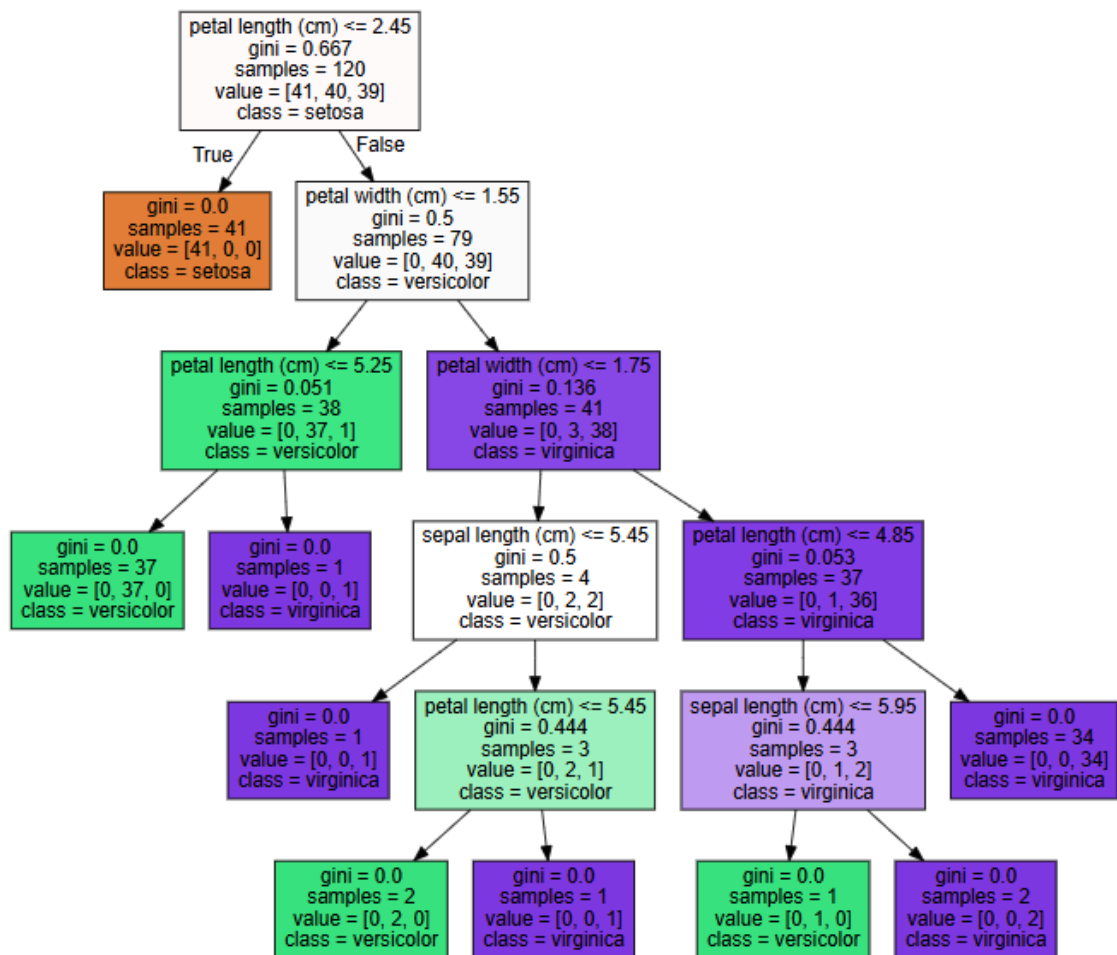


```
DecisionTreeClassifier(random_state=156)
```

```
from sklearn.tree import export_graphviz

export_graphviz(dt_clf, out_file='tree.dot', class_names = iris_data.target_names,
                feature_names = iris_data.feature_names, impurity=True, filled=True)
```

```
import graphviz
with open('tree.dot') as f:
    dot_graph = f.read()
graphviz.Source(dot_graph)
```



- `feature_importances_` : 피처가 트리 분할 시 정보 이득이나 지니 계수를 얼마나 효율적으로 잘 개선시켰는지를 정규화된 값으로 표현한 것
 - ↳ ndarray 형태로 값을 반환하며 피처 순서대로 값이 할당
 - ↳ 일반적으로 값이 높을수록 해당 피처의 중요도 ↑

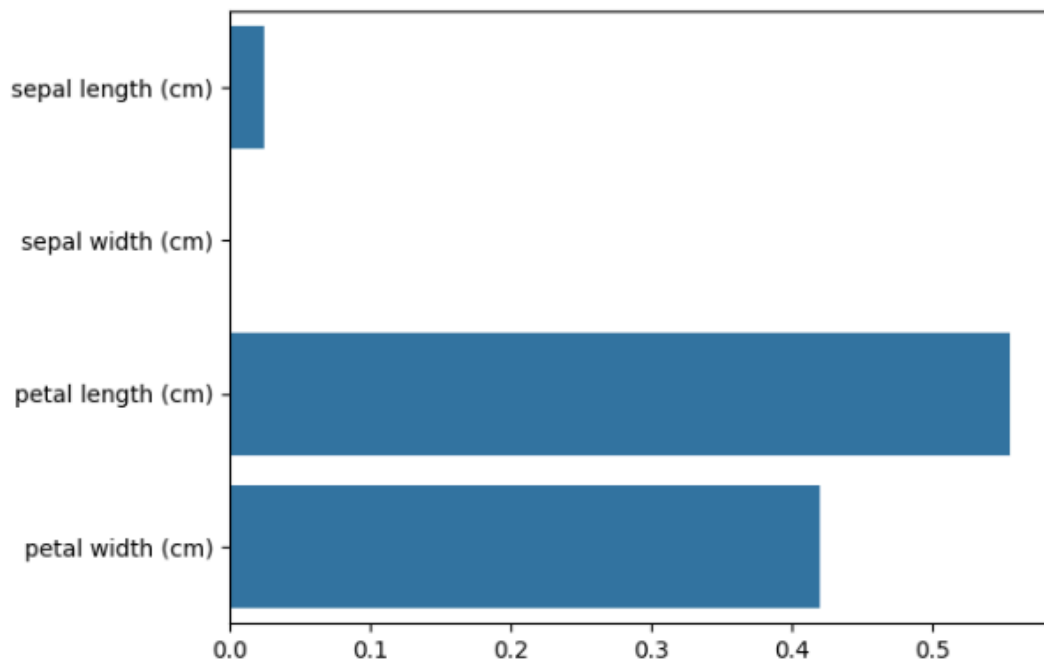
```
import seaborn as sns
import numpy as np
%matplotlib inline

print('Feature importances: #n{}'.format(np.round(dt_clf.feature_importances_, 3)))

for name, value in zip(iris_data.feature_names, dt_clf.feature_importances_):
    print('{} : {:.3f}'.format(name, value))

sns.barplot(x=dt_clf.feature_importances_, y=iris_data.feature_names)
```

```
Feature importances:
[0.025 0.    0.555 0.42 ]
sepal length (cm) : 0.025
sepal width (cm) : 0.000
petal length (cm) : 0.555
petal width (cm) : 0.420
<Axes: >
```



- petal_length의 피쳐 중요도가 가장 높음

결정 트리 과적합

- `make_classification()` : 분류를 위한 테스트용 데이터를 쉽게 만들어주는 함수

```

from sklearn.datasets import make_classification
import matplotlib.pyplot as plt
%matplotlib inline

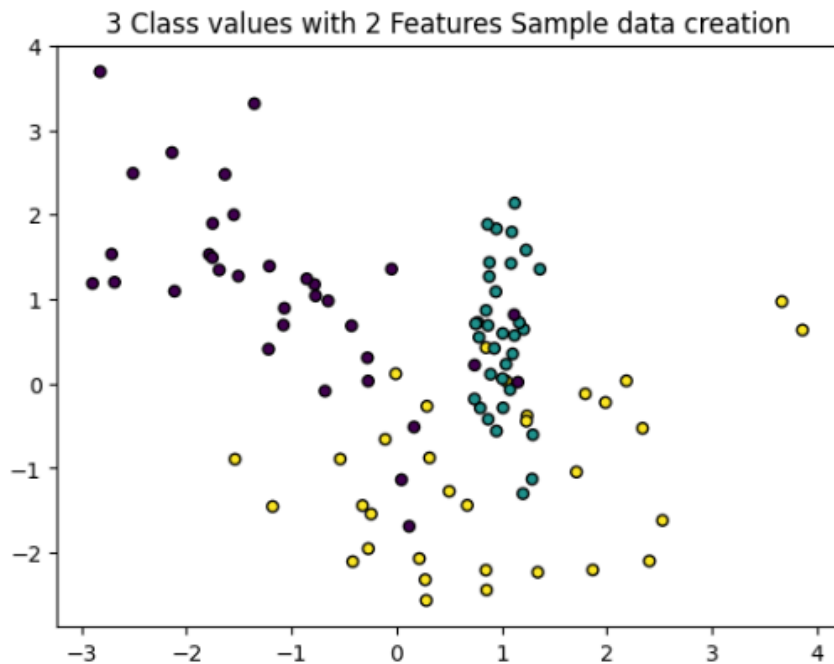
plt.title('3 Class values with 2 Features Sample data creation')

X_features, y_labels = make_classification(n_features=2, n_redundant=0, n_informative=2,
                                          n_classes=3, n_clusters_per_class=1, random_state=0)

plt.scatter(X_features[:, 0], X_features[:, 1], marker='o', c=y_labels, s=25, edgecolor='k')

<matplotlib.collections.PathCollection at 0x7d9c8cb4cc20>

```



앙상블 학습

- 앙상블 학습: 여러 개의 분류기를 생성하고 그 예측을 결합함으로써 보다 정확한 최종 예측을 도출하는 기법

- 단일 분류기보다 신뢰성이 높음

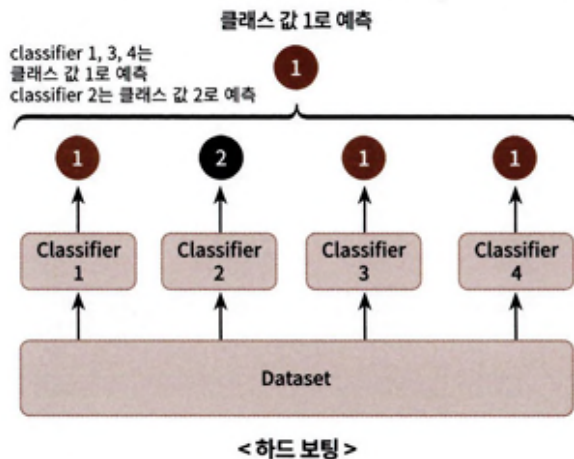
유형	설명	예시
보팅	서로 다른 알고리즘을 가진 분류기에 대한 투표를 통해 최종 예측 결과를 결정	-
배깅	서로 같은 유형의 알고리즘을 기반으로 학습을 수행해 보팅	랜덤 포레스트

유형	설명	예시
부스팅	순차적으로 학습하되 앞에서 틀린 데이터에 대해 가중치를 부여하며 학습과 예측을 진행	그래디언트 부스팅, XGBoost, LightGMB 등

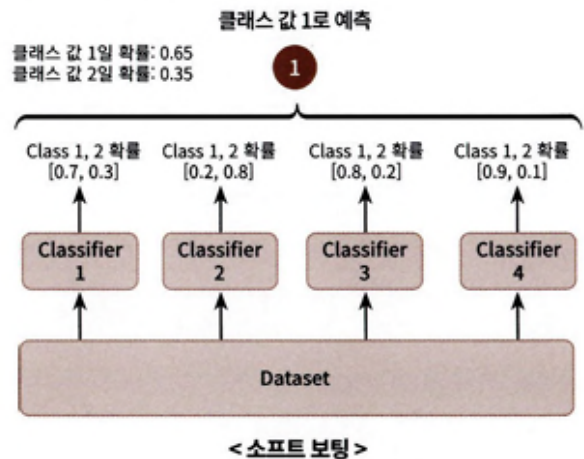
보팅 유형 - 하드 보팅과 소프트 보팅

- **하드 보팅** : 예측한 결과값들 중 다수의 분류기가 결정한 예측값을 최종 보팅 결과값으로 선정(= 다수결 원칙)
- **소프트 보팅** : 분류기들의 레이블 값 결정 확률을 평균해서 이들 중 확률이 가장 높은 레이블 값을 최종 보팅 결과값으로 선정

Hard Voting은 다수의 classifier 간 다수결로 최종 class 결정



Soft Voting은 다수의 classifier들의 class 확률을 평균하여 결정



보팅 분류기

```
import pandas as pd

from sklearn.ensemble import VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

cancer = load_breast_cancer()

data_df = pd.DataFrame(cancer.data, columns=cancer.feature_names)
data_df.head(3)
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	radius error
0	17.99	10.38	122.8	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	1.0950
1	20.57	17.77	132.9	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	0.5436
2	19.69	21.25	130.0	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	0.7456

```
lr_clf = LogisticRegression(solver='liblinear')
knn_clf = KNeighborsClassifier(n_neighbors=8)

vo_clf = VotingClassifier(estimators=[('LR', lr_clf), ('KNN', knn_clf)], voting='soft')

X_train, X_test, y_train, y_test = train_test_split(cancer.data, cancer.target,
                                                    test_size=0.2, random_state=156)

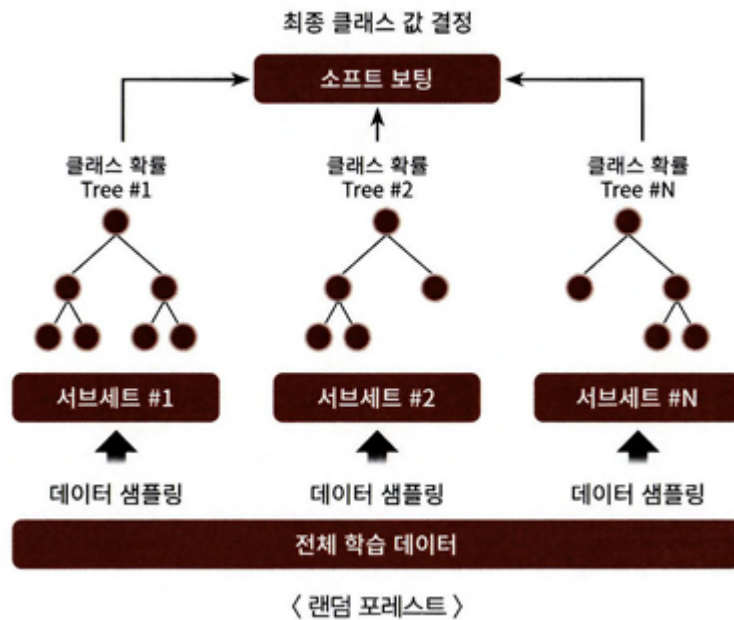
vo_clf.fit(X_train, y_train)
pred = vo_clf.predict(X_test)
print('Voting 분류기 정확도: {:.4f}'.format(accuracy_score(y_test, pred)))

classifiers = [lr_clf, knn_clf]
for classifier in classifiers:
    classifier.fit(X_train, y_train)
    pred = classifier.predict(X_test)
    class_name = classifier.__class__.__name__
    print('{} 정확도: {:.4f}'.format(class_name, accuracy_score(y_test, pred)))
```

```
Voting 분류기 정확도: 0.9561
LogisticRegression 정확도: 0.9474
KNeighborsClassifier 정확도: 0.9386
```

랜덤 포레스트

- 배경의 대표적인 알고리즘
- 앙상블 알고리즘 중 비교적 빠른 수행 속도
- 다양한 영역에서 높은 예측 성능을 보임
- 결정 트리 기반 알고리즘



- 부트스트래핑: 여러 개의 데이터 세트를 중첩되게 분리하는 것

```
# 랜덤포레스트
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
import pandas as pd
import warnings
warnings.filterwarnings('ignore')

X_train, X_test, y_train, y_test = get_human_dataset()

rf_clf = RandomForestClassifier(random_state=0, max_depth=8)
rf_clf.fit(X_train, y_train)
pred = rf_clf.predict(X_test)
accuracy = accuracy_score(y_test, pred)
print('랜덤 포레스트 정확도: {:.4f}'.format(accuracy))
```

랜덤 포레스트 정확도: 0.9220

랜덤 포레스트 하이퍼 파라미터 및 튜닝

- `n_estimators` : 랜덤 포레스트에서 결정 트리의 개수를 지정
디폴트는 10개이고 많이 설정할수록 좋은 성능을 기대할 수 있지만, 성능이 무조건 향상되지는 않음 + 학습 수행 시간 ↑
- `max_features` : 최적의 분할을 위해 고려할 최대 피쳐 개수
디폴트는 'auto'이기 때문에 $\sqrt{\text{전체 피쳐 개수}}$ 만큼 참조
- `max_depth`, `min_samples_leaf`, `min_samples_split`

```

from sklearn.model_selection import GridSearchCV

params = {
    'max_depth': [8, 16, 24],
    'min_samples_leaf': [1, 6, 12],
    'min_samples_split': [2, 8, 16]
}

rf_clf = RandomForestClassifier(n_estimators=100, random_state=0, n_jobs=-1)
grid_cv = GridSearchCV(rf_clf, param_grid=params, cv=2, n_jobs=-1)
grid_cv.fit(X_train, y_train)

print('최적 하이퍼 파라미터: \n', grid_cv.best_params_)
print('최고 예측 정확도: {:.4f}'.format(grid_cv.best_score_))

```

최적 하이퍼 파라미터:
 {'max_depth': 16, 'min_samples_leaf': 6, 'min_samples_split': 16}
 최고 예측 정확도: 0.9157

```

rf_clf1 = RandomForestClassifier(n_estimators=100, min_samples_leaf=6, max_depth=16,
                                min_samples_split=2, random_state=0)
rf_clf1.fit(X_train, y_train)
pred = rf_clf1.predict(X_test)
print('예측 정확도: {:.4f}'.format(accuracy_score(y_test, pred)))

```

예측 정확도: 0.9253



조윤서



이전 포스트

[Euron] 머신러닝 스터디 2주차

0개의 댓글

댓글을 작성하세요

댓글 작성

